



Pontificia Universidad Javeriana
Departamento de Ingeniería de Sistemas
Estructuras de Datos
Taller 1: complejidad, 2026-30

1 Objetivo

Analizar la ejecución de los algoritmos:

- Cálculo de la suma de los múltiplos de un número k menores o iguales a un número n , implementado en versión recursiva, iterativa y mediante una fórmula matemática (tiempo constante).
- Ordenamiento de un vector de n elementos (bubblesort, quicksort y heapsort).

2 Desarrollo del taller

El desarrollo del taller consistirá en generar un informe escrito (en formato .pdf, **cualquier otro formato implica una nota final de 0.0/5.0**), el cual debe enviarse a través de la respectiva actividad de BRIGHTSPACE a la hora estipulada en la actividad.

Junto con este enunciado se entrega un archivo comprimido que contiene el conjunto de programas base que deberán utilizarse para el desarrollo del taller.

Este informe debe seguir el formato entregado al inicio del curso (Formato Springer, **cualquier otro formato implica una nota final de 0.0/5.0**).

Dado que el formato proporcionado contiene títulos genéricos para el desarrollo del proyecto del curso, para este taller el estudiante deberá reemplazar dichos títulos por los títulos definidos en este enunciado, desarrollando cada una de las preguntas de manera individual inmediatamente después de cada título correspondiente.

2.1 (15%) Análisis inicial

Estudie los archivos de código fuente que se adjuntan (*h, *.hxx, *.cxx). Responda en la primera parte de su reporte las siguientes preguntas:

1. (10%) Organización del código

- ¿Por qué el proyecto está dividido en múltiples archivos de código y no implementado en un único archivo? Explique las ventajas de esta organización modular, considerando aspectos como la legibilidad del código, la reutilización, la separación entre definición e implementación, y la facilidad de compilación y mantenimiento. No se centre únicamente en este proyecto sino consulte en fuentes de información por qué se recomienda esta separación y estructuración de un proyecto de software de C++.

2. (25%) Tipos de archivos de código

- ¿Cuál es la diferencia entre un archivo con extensión ".cxx", uno con extensión ".h" y uno con extensión ".hxx"? Describa el propósito de cada tipo de archivo: ¿Qué suele contener un archivo ".h"? ¿Qué diferencia hay entre un ".h" y un ".hxx" en este proyecto? ¿Qué rol cumple un archivo ".cxx"? Finalmente, describa cómo se relacionan estos archivos entre sí durante el proceso de compilación de los programas.

3. (35%) Archivos de prueba

- Los programas escritos en los archivos "test_*.cxx" ¿qué hacen? Identifique cuáles archivos comienzan con "test_" y analice su contenido. ¿Qué funcionalidad prueba cada uno? ¿Qué datos de entrada utilizan y qué salida generan? Explique brevemente cuál es el propósito de tener estos archivos en el proyecto y cómo contribuyen al análisis de los algoritmos.

La secuencia de comandos que se debe usar en este taller está en el archivo “comandos.txt”. Analice los comandos allí presentes, identifique la sintaxis de cada uno e intente comprender qué resultado genera cada uno. Para esto, copie y pegue cada comando en la consola de comandos y espere los resultados (algunos pueden tardar varios segundos o minutos).

4. (10%) Tipos de comandos y sintaxis

- El archivo contiene tres tipos generales de comandos, ¿cuáles son esos tipos? Para cada clasificación, describa detalladamente el funcionamiento de los comandos, explicando qué hace cada parte del comando (por ejemplo: nombre del ejecutable, parámetros, redirecciones, etc.). Dado que existen comandos de ejecución diferentes según el sistema operativo en el cual el estudiante ejecute el taller, el análisis deberá centrarse únicamente en los comandos utilizados para el sistema operativo correspondiente, sin necesidad de describir aquellos destinados a otros entornos.

5. (10%) Archivos generados

- Una vez haya ejecutado todos los comandos del archivo “comandos.txt”, identifique los nuevos archivos que aparecieron en la carpeta del proyecto. Enumérelos y explique brevemente qué sugiere su nombre y/o extensión (por ejemplo, si corresponden a tablas, resultados, salidas de prueba, etc.).

6. (10%) Contenido y propósito de los archivos

- Analice los archivos generados e indique qué parecen contener o representar. Explique qué tipo de información almacenan (por ejemplo: resultados numéricos, tablas, datos sin procesar, etc.) y cómo se relacionan con los comandos ejecutados y con los algoritmos que se están analizando.

2.2 (40%) Cálculo de la suma de los múltiplos de un número k menores o iguales a un número n

La primera parte del archivo “comandos.txt” se refiere al análisis del algoritmo que calcula la suma de todos los múltiplos de un número que sean menores o iguales a un número, implementado en tres versiones: recursiva, iterativa y mediante una fórmula matemática (versión constante).

Para este propósito, se utiliza el programa compilado “my_test_suma”, el cual recibe como parámetros:

- El valor de “ n ”, que representa el límite superior hasta donde se desea realizar la suma.
- El valor de “ k ”, que representa el número base cuyos múltiplos se deben sumar.

El programa genera una salida compuesta por 8 campos, separados por punto y coma, en el siguiente orden:

- El valor de “ n ”,
- El valor de “ k ”,
- El resultado de la versión recursiva,
- El resultado de la versión iterativa,
- El resultado de la versión constante,
- El tiempo de ejecución del algoritmo recursivo,
- El tiempo de ejecución del algoritmo iterativo,
- El tiempo de ejecución del algoritmo constante.

Este programa puede ejecutarse en la consola de la siguiente manera:

```
$ ./my_test_suma  $n$   $k$ 
```

Por ejemplo, para calcular la suma de los múltiplos de 2 menores o iguales a 5000:

```
$ ./my_test_suma 5000 2
```

Antes de realizar el análisis empírico de los tiempos de ejecución, el estudiante deberá:

1. (20%) Demostración e implementación de la versión constante

- Demostrar matemáticamente cómo es posible calcular directamente la suma de los múltiplos de un número “k” menores o iguales a “n”, utilizando una fórmula cerrada. Describa claramente el razonamiento en su informe e implemente esta solución dentro del código fuente, en el lugar indicado de la función correspondiente a la versión constante (Función “suma_constant”).

Una vez implementada y verificada esta versión constante, se procede con el análisis empírico. Los comandos incluidos en el archivo “comandos.txt” permiten automatizar la ejecución del programa para distintos valores de “n”, manteniendo “k” fijo en diferentes valores (por ejemplo 2, 3 y 10), y redirigiendo los resultados a archivos CSV (“suma_table_k2.csv”, “suma_table_k3.csv”, “suma_table_k10.csv”).

Luego de ejecutar los comandos, el estudiante deberá importar los archivos generados en Excel para construir tres gráficas comparativas, una por cada valor de “k” utilizado. En cada gráfica:

- El eje X debe representar el valor de “n”.
- El eje Y debe representar el tiempo de ejecución.
- Deben aparecer tres curvas correspondientes a los tres algoritmos: recursivo, iterativo y constante.

Estas gráficas permitirán observar, comparar y analizar el comportamiento relativo de los algoritmos según el tamaño de entrada.

2. (20%) Análisis gráfico

- Examine cada una de las tres gráficas generadas. ¿Cuál de los algoritmos presenta un crecimiento más acelerado en el tiempo de ejecución conforme aumenta “n”? ¿Cuál mantiene un tiempo estable? ¿Existen diferencias en la forma de las curvas según el valor de “k”? Justifique su análisis únicamente con base en la información visible en las gráficas.

3. (25%) Comparación de resultados y correctitud

- Compare los valores obtenidos por las tres versiones del algoritmo. ¿Coinciden los resultados numéricos en todos los casos? ¿Qué sucede cuando “n” es muy grande? ¿Se presentan errores por desbordamiento? En caso afirmativo, identifique en qué versiones ocurre y proponga una solución razonada. Para esta última parte, ejecute el programa con los valores “n” de 65535 y 65536 utilizando los mismos valores de “k” empleados en los análisis previos. ¿Qué resultados se obtienen? ¿Son todos correctos? En caso de errores, analice por qué ocurren y proponga una o varias modificaciones en el código para evitar este comportamiento. Documente claramente los cambios realizados y justifique su implementación.

4. (25%) Relación con la eficiencia teórica

- Determine, a partir del análisis del código fuente, la eficiencia teórica esperada (en notación O) de cada una de las tres versiones del algoritmo. Relacione esta eficiencia teórica con el comportamiento observado en la gráfica empírica y argumente si los resultados obtenidos concuerdan o no con lo esperado.

5. (10%) Reflexión final

- En función de los resultados obtenidos, ¿qué versión del algoritmo recomendaría usar en un programa real? ¿Dependería su recomendación del rango esperado de valores de ? Justifique su respuesta con base en evidencia empírica y teórica

2.3 (45%) Sort

La segunda parte del archivo “comandos.txt” se refiere al análisis de tres algoritmos de ordenamiento diferentes (*bubblesort*, *quicksort* y *heapsort*), ejecutados sobre arreglos en tres configuraciones diferentes: arreglos completamente desordenados, arreglos ordenados y arreglos ordenados “al contrario”.

Para esto, se utilizan los programas compilados “my_test_sort_random”, “my_test_sort_sorted” y “my_test_sort_inverse”, los cuales reciben como único parámetro el tamaño del arreglo a ordenar, y generan cada uno una salida con 7 valores: el tamaño del arreglo, la verificación de ordenamiento con cada uno de los algoritmos (si el

arreglo sí queda ordenado o no al finalizar la ejecución), y el tiempo de ejecución de cada algoritmo. Cada uno de estos programas puede ejecutarse individualmente en la consola así:

```
$ ./my_test_sort_random tamano_arreglo
$ ./my_test_sort_sorted tamano_arreglo
$ ./my_test_sort_inverse tamano_arreglo
```

Para realizar el análisis empírico de la complejidad de los tres algoritmos de ordenamiento en las tres configuraciones de arreglos, cada programa se ejecuta varias veces con diferentes tamaños de arreglo (entre 1000 y 20000, en este caso) para generar una tabla para cada algoritmo con los respectivos valores de ejecución ("sort_random_table.csv", "sort_sorted_table.csv" y "sort_inverse_table.csv").

Luego, el estudiante debe importar cada archivo CSV en Excel y generar una gráfica por configuración de entrada (aleatoria, ordenada, inversa). En cada gráfica:

- El eje X debe representar el tamaño del arreglo ("n").
- El eje Y debe representar el tiempo de ejecución (en milisegundos).
- Se deben graficar las tres curvas correspondientes a bubblesort, quicksort y heapsort en el mismo gráfico para facilitar la comparación.

Estas gráficas deben ser elaboradas por el estudiante en su informe, no se entregan generadas por el sistema. La interpretación de estas curvas será clave para el análisis solicitado.

A partir de las gráficas y observaciones realizadas, el estudiante deberá incluir en su informe las respuestas a las siguientes preguntas:

1. (45%) Análisis empírico inicial

- Examine las tres gráficas generadas, correspondientes a las configuraciones de entrada aleatoria, ordenada e inversa. Para cada gráfica, describa el comportamiento observado de los algoritmos bubblesort, quicksort y heapsort. ¿Cuál algoritmo presenta el mejor desempeño en cada caso? ¿Cuál muestra un crecimiento más acelerado conforme aumenta el tamaño del arreglo? Fundamente su respuesta únicamente con base en la observación de las gráficas, sin considerar aún la complejidad teórica.

2. (15%) Complejidad teórica: mejor, promedio y peor caso

- Investigue la complejidad teórica esperada (en notación O) de los algoritmos bubblesort, quicksort y heapsort, considerando explícitamente los tres escenarios clásicos: mejor caso, caso promedio, peor caso. Resuma esta información en una tabla comparativa, indicando claramente la complejidad correspondiente a cada algoritmo en cada uno de estos tres casos. Indique la fuente bibliográfica o documentación consultada.

3. (40%) Relación entre teoría y observación empírica

- Reanalice las gráficas generadas a la luz de las complejidades teóricas investigadas. Relacione cada tipo de entrada (aleatoria, ordenada e inversa) con el mejor caso, caso promedio o peor caso de cada algoritmo, según corresponda. ¿El comportamiento empírico observado en las gráficas concuerda con las complejidades teóricas esperadas? En caso de discrepancias, explique qué factores podrían justificarlas (por ejemplo, implementación específica, constantes ocultas, efectos de caché, tamaño de entrada, entre otros).

3 Evaluación

La entrega se hará a través de la correspondiente actividad de BrightSpace, antes de la media noche del **sábado 08 de agosto de 2026**. Se debe entregar un único archivo con el informe (**único formato aceptado: .pdf**). Si la entrega contiene archivos en cualquier otro formato, será descartada y no será evaluada, es decir, la nota definitiva de la entrega será de 0 (cero) sobre 5 (cinco).

1. **(60%) Resultado del trabajo en grupo (informe):** el resultado consignado en el informe se evaluará con la siguiente escala para cada uno de los puntos:
 - **Excelente (5.0/5.0):** El estudiante presenta análisis y datos organizados (información) que le permiten llegar a una conclusión coherente sobre la comparación de los algoritmos. Además, el estudiante es capaz de explicar la coherencia de los resultados.
 - **Bueno (4.0/5.0):** El estudiante tiene una idea de cómo comparar los algoritmos e interpretar los resultados, pero falla en el formalismo. Además, no presenta datos organizados o no hizo un análisis de la coherencia de los resultados.
 - **Regular (3.0/5.0):** El estudiante logra esbozar una idea de comparación de los algoritmos e interpretación de los resultados, pero falla en presentarla de manera clara y contundente. No presenta datos organizados ni su correspondiente análisis.
 - **Malo (1.5/5.0):** El estudiante no logra entender las instrucciones presentadas, y su análisis es vago o ambiguo.
 - **No entregó (0.0/5.0):** No entregó el archivo del informe, o el archivo entregado no está en formato PDF.
2. **(40%) Evaluación individual del resultado del taller:** cada estudiante, de forma individual, deberá responder algunas preguntas sobre el trabajo realizado. Cada pregunta se evaluará así:
 - **Excelente (5.0/5.0):** La respuesta es correcta y concuerda totalmente con la entrega realizada por el grupo de trabajo.
 - **Bueno (3.0/5.0):** La respuesta es parcialmente correcta y/o concuerda sólo en algunos aspectos con la entrega realizada por el grupo de trabajo.
 - **Malo (1.0/5.0):** La respuesta no es correcta y/o no concuerda de ninguna forma con la entrega realizada por el grupo de trabajo.
 - **No entregó (0.0/5.0):** El estudiante no entregó una respuesta para la pregunta.

3.1 Recomendaciones: uso de herramientas de IA generativa

Para la implementación de los componentes de este taller, se espera que los estudiantes evidencien la apropiación y aplicación de los conceptos estudiados en clase. No obstante, se permite el uso de herramientas de inteligencia artificial generativa (como ChatGPT, GitHub Copilot, Copilot Chat, entre otras), bajo las siguientes condiciones:

1. **Identificación clara:** Todo fragmento de código, texto, explicación o análisis que haya sido generado con el apoyo de una herramienta de IA generativa debe estar explícitamente identificado mediante un comentario o nota aclaratoria en el informe o código fuente.
2. **Referencia del prompt:** Si se genera un contenido textual o de código completo con IA, se debe incluir el prompt (instrucción) utilizado para obtener dicho contenido, ya sea directamente o en un comentario adjunto al fragmento generado.
3. **Transformaciones propias:** Si el contenido generado por IA ha sido posteriormente editado, modificado o adaptado por el estudiante, esta transformación también debe dejarse explícita, indicando qué se cambió o adaptó y con qué fin.
4. **Límite de contenido generado:** Se sugiere que el contenido generado por herramientas de IA no supere un tercio del total del trabajo entregado, de manera que se garantice el dominio propio del estudiante sobre los conceptos desarrollados.

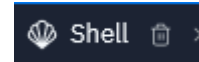
El no cumplimiento de estas recomendaciones podrá ser considerado como falta de honestidad académica si se detecta un uso encubierto de herramientas automatizadas en la elaboración del trabajo.

4 Ayuda con Repl-it

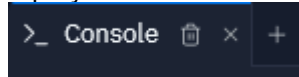
Como bien se comentó en clase, una herramienta muy útil para los talleres es la página <https://replit.com> donde se encuentra un ambiente de desarrollo provisto para programar en C++ y en otros lenguajes. Ya que algunos de los estudiantes optan por esta herramienta para el desarrollo de este taller se provee esta sección para la ejecución correcta del taller en cuestión.

4.1 Ventana de terminal (SHELL)

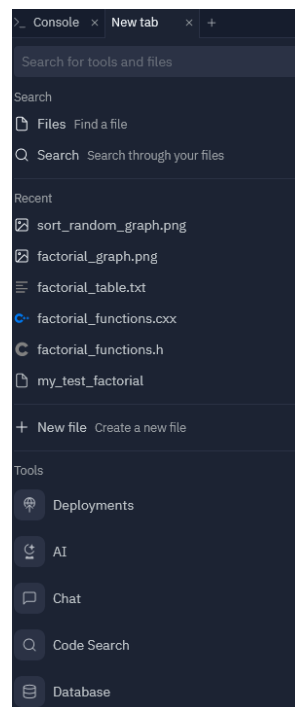
Para ejecutar los comandos se debe hacer en la ventana que se llama **SHELL**. Esta ventana la puede encontrar a mano izquierda de su proyecto. Si no la encuentra puede adicionarla con el símbolo + que se encuentra



al final de la barra de ventanas



Al darle click a este símbolo aparece la siguiente ventana donde buscará **SHELL** y lo seleccionara.



4.2 Pegar comandos en la ventana SHELL

Para pegar comandos en esta herramienta se debe hacer a través de la siguiente combinación de teclas “**CTRL + SHIFT + V**”